

EduVerify — Azure Deployment Plan

Document: Production deployment architecture for Azure
Application: EduVerify Pre-Verification Portal (Node.js + PostgreSQL)
Scale target: ~5,000 students (admission season), scalable to 25,000+
Prepared: June 2026

Executive summary: Deploy EduVerify as a single Node.js web application on **Azure App Service (Linux)**, with **Azure Database for PostgreSQL Flexible Server** for all student and verification data. Document files can remain on **Cloudinary** for the fastest migration, or move to **Azure Blob Storage** for full in-Azure compliance. This architecture supports real-time orientation-week verification with room to grow.

1. Application overview

Component	Current technology	Azure service
Web application (API + UI)	Node.js 20, Express, static HTML/React	Azure App Service (Linux)
Relational database	PostgreSQL	Azure Database for PostgreSQL — Flexible Server
Document storage	Cloudinary (authenticated files)	Cloudinary or Azure Blob Storage
Secrets & config	Environment variables	Azure Key Vault + App Service references
HTTPS & domain	Render URL	Custom domain + managed certificate

Monitoring	Platform logs	Application Insights + Log Analytics
------------	---------------	--------------------------------------

EduVerify is a **monolithic** application: one deployable unit serves `/api/*`, the student portal (`index.html`), and the verification schedule dashboard (`verify.html`).
Kubernetes is not required at 5,000-student scale.

2. Recommended production architecture

[Students + Verification Staff] | Azure DNS (college domain) | Azure Front Door (optional: WAF + CDN) | Azure App Service – Linux, Node.js 20 (always on, auto-scale 1–3 instances) | +-----+-----+-----+ | | PostgreSQL Flexible Azure Key Vault Cloudinary OR Server (private/VNet) JWT, DB password, Azure Blob Storage admin secrets (student documents)

3. Capacity planning — 5,000 students

3.1 Database (PostgreSQL)

Data type	Estimated volume
Student records	~5,000 rows (growing yearly)
Document metadata	50,000–75,000 rows (10–15 docs per student)
Verification schedule slots	50,000–100,000 rows (flexible dates/rooms)
Audit log	5–20 GB over one academic cycle

Total DB size (metadata only)	2–8 GB initially
-------------------------------	-------------------------

Recommended starting tier: Azure Database for PostgreSQL Flexible Server — **General Purpose, 2 vCores, 32 GB storage**. Enable automated backups (7–35 days). Use zone-redundant HA if college policy requires it.

Growth (10K–25K students): Scale to 4 vCores, 64–128 GB storage.

3.2 Application server

Setting	Recommendation
App Service plan	Premium v3 P1v3 (2 vCPU, 8 GB RAM) for production
Instances	Minimum 1; auto-scale to 2–3 during orientation peaks
Always On	Enabled (required for Node.js)
Peak concurrency	Plan for 200–500 concurrent users, not all 5,000 at once

3.3 Document file storage

Option	When to use	Estimated size (5K students)
Cloudinary	Fastest migration from Render; no code changes	50–150 GB

Azure Blob Storage	All data must stay in Azure; long-term archive	100–500 GB multi-year
---------------------------	---	-----------------------

4. Azure resources checklist

Create these in a dedicated resource group (e.g. rg-eduverify-prod):

1. **Resource Group** — rg-eduverify-prod
2. **App Service Plan** — Linux, Premium v3
3. **Web App** — eduverify-prod
4. **PostgreSQL Flexible Server** — eduverify-pg-prod
5. **Key Vault** — store JWT, database URL, admin password, Cloudinary keys
6. **Storage Account** — optional, for Azure Blob documents
7. **Application Insights** — performance and error monitoring
8. **Log Analytics Workspace** — centralised logs
9. **Azure Front Door** — optional WAF and global access
10. **Private Endpoint** — optional, database accessible only inside VNet

5. Security configuration

- **PostgreSQL:** Disable public access where possible; use VNet integration or Private Endpoint. Require SSL (`sslmode=require`).
- **HTTPS:** Enforce HTTPS only on App Service; TLS 1.2 minimum.
- **CORS:** Set `CORS_ORIGIN` to exact production URL (never * in production).
- **JWT_SECRET:** Minimum 32 random characters; stored in Key Vault.
- **Admin password:** Strong `SEED_ADMIN_PASSWORD` in Key Vault; production syncs to ADM-001 on deploy.
- **Managed Identity:** App Service reads secrets from Key Vault without storing them in code.
- **Region:** Deploy in Central India or institution-approved Azure region for data residency.

6. Required environment variables

Variable	Purpose
----------	---------

NODE_ENV	production
DATABASE_URL	PostgreSQL connection string with SSL
JWT_SECRET	Token signing secret (48+ byte random)
JWT_EXPIRES_IN	12h (session duration)
CORS_ORIGIN	https://eduverify.yourcollege.edu
CLOUDINARY_CLOUD_NAME, API_KEY, API_SECRET	Document storage (if using Cloudinary)
SEED_ADMIN_ID	ADM-001
SEED_ADMIN_PASSWORD	Supervisor login password
AUTO_SETUP	true on first deploy (schema + admin seed)

7. CI/CD deployment pipeline

Phase 1 — Manual pilot

1. Connect GitHub repository to Azure App Service (Deployment Center).
2. Build: `npm ci --omit=dev`
3. Start command: `npm start`

4. First boot runs database setup automatically.

Phase 2 — Automated production

```
Git push (main branch) → GitHub Actions or Azure DevOps → npm ci → Deploy to App Service staging slot → Smoke test GET /api/health → Swap staging → production (zero-downtime)
```

8. Migration from Render

Step	Action
1	Export PostgreSQL from Render using pg_dump
2	Create Azure PostgreSQL and import the dump
3	Deploy application to App Service with Azure DATABASE_URL
4	Keep same Cloudinary account (documents unchanged) or migrate to Blob
5	Update DNS to Azure custom domain
6	Update CORS_ORIGIN to new URL
7	Test full workflow: login, upload, declaration, verification, reassign
8	Decommission Render after successful cutover

Planned downtime: 1–4 hours for database cutover, or run parallel systems for one day.

9. Scaling roadmap

Stage	Students	App Service	PostgreSQL	Storage
Pilot	500	B2, 1 instance	Burstable 2 vCore, 32 GB	Cloudinary
Production	5,000	P1v3, scale 1–3	GP 2 vCore, 32–64 GB	100+ GB
Growth	10,000–25,000	P2v3, scale 2–5	GP 4 vCore, 128 GB	Blob + CDN
Multi-year	50,000+	Container Apps / AKS	Partition by admission year	Archive tier

10. Estimated monthly cost (India region, indicative)

Service	Pilot	Production (5K students)
App Service P1v3	—	■8,000 – ■12,000
PostgreSQL GP 2 vCore	—	■10,000 – ■18,000
Storage / Cloudinary	■0 – ■2,000	■2,000 – ■8,000
Application Insights	■500 – ■2,000	■1,000 – ■3,000
Total (approximate)	■3,000 – ■8,000	■20,000 – ■40,000

Use the Azure Pricing Calculator for exact quotes. College EA, CSP, or Azure for Students programmes may reduce cost.

11. Rollout phases

Phase A — Foundation (1–2 weeks)

- Create Azure subscription, resource group, PostgreSQL, App Service, Key Vault
- Deploy EduVerify from GitHub; configure SSL and custom domain
- Set security: JWT, CORS, firewall, strong admin password

Phase B — Data & load test (1 week)

- Bulk upload 5,000 student records via admin Excel template
- Load test: concurrent uploads and live verification dashboard
- Backup and restore drill

Phase C — Go-live (admission season)

- Staff training (supervisor + verifiers)
- Monitor Application Insights during orientation week
- Auto-scale enabled; support runbook ready

Phase D — Hardening (post go-live)

- Migrate documents to Azure Blob if required by IT policy
- Private endpoints for database; WAF via Front Door
- Archive prior-year admission data

12. Key decisions

Question	Recommendation
App hosting	Azure App Service (Linux, Node 20)

Database	Azure Database for PostgreSQL Flexible Server
Documents	Cloudinary first; Azure Blob when IT requires all-Azure storage
High availability	Zone-redundant DB + App Service scale-out for peaks
Kubernetes?	Not needed for 5,000 students unless mandated by policy
Azure region	Central India (or compliance-approved region)

13. Prerequisites from college IT

1. Azure subscription (pay-as-you-go or Enterprise Agreement)
2. DNS hostname for the portal (e.g. eduverify.college.edu)
3. Decision: Cloudinary vs Azure Blob for student documents
4. Data retention policy for uploads and audit logs
5. Network rules if database must be campus-only
6. Maintenance window for Render-to-Azure cutover

